Deliverable 3: Testing, Security, Performance, Cost, Deployment and Reliability

CS455: Introduction to Software Engineering · Instructor: Prof. Priyanka Bagade

Concord: A Collaborative Code Execution and Debugging Platform

Pallav Rastogi (230731) · Aamir Ahmad (230010) · Aryamann Srivastava (230211) · Tattwa Shivani (231089) · Sanchit Arora (230907)

Deliverable 2 delivered the implementation. This submission tests it, attacks it, measures it, costs it, and records how it is deployed and how it fails. Every number in it was measured on 9f169d8, the head of `main` of `https://github.com/aamirepe/Concord---CS455-Course-Project` on 2 November 2026, or read from Jira and GitHub on that date. On 4 November the same Jira export and the repository head were read again: no ticket status or assignee had changed and `main` was still at 9f169d8. Section 1.3 maps each item of the Deliverable 3 brief to the section that answers it.

Contents

1 Scope, Method and Basis of the Evidence **3**
1.1 What was run, where, and by whom . . . 3
1.2 How the measurements were isolated . . . 3
1.3 Contents of the brief and where each item is answered . . . 3

2 Unit, Integration, System and Adversarial Test Report **3**
2.1 Results . . . 4
2.2 Tests by level . . . 4
2.3 Static analysis and builds . . . 4
2.4 Test inventory . . . 5

3 Code Coverage Report **6**

4 Concurrency Test Report **6**
4.1 Tests and outcomes . . . 7
4.2 Live concurrency measurements . . . 7

5 Security and Threat-Model Report **7**
5.1 Assets, actors and trust boundaries . . . 7
5.2 Threat model . . . 8
5.3 Scans . . . 9

6 Performance, Load Test and Optimisation Report **9**
6.1 Requirements against measurements . . . 9
6.2 Experiments E1 to E5 . . . 10
6.3 Optimisation results . . . 10
6.4 Resource use . . . 10

7 Failure and Recovery Test Report **10**
7.1 Scenarios, mechanisms and results . . . 11
7.2 The worker-kill run in detail . . . 11

8 Defect History and Evidence of Fixes **11**
8.1 How defects were found . . . 12
8.2 Defects found and fixed . . . 12
8.3 Bug and defect history in Jira . . . 13

9 Costing Document **13**
9.1 Sizing from measurements . . . 14
9.2 Monthly operating cost . . . 14

9.3 LLM cost per request and its ceiling . . . 14
9.4 Cost per run . . . 14
9.5 Scenarios . . . 15
9.6 Development effort . . . 15

10 Timeline and Gantt Chart **15**
10.1 Plan against what happened . . . 16

11 Deployment Documentation **16**
11.1 Architecture of the production deployment . . . 16
11.2 Prerequisites . . . 16
11.3 Configuration . . . 17
11.4 First deployment and updates . . . 17
11.5 Automated deployment . . . 17
11.6 Health, logs and monitoring . . . 17
11.7 Backup and restore . . . 17
11.8 Rollback . . . 17

12 Final Requirements Traceability Matrix **18**
12.1 Functional requirements . . . 18
12.2 Non-functional requirements . . . 19
12.3 Worked chains for the critical requirements . . . 22
12.4 Jira task to pull request . . . 22

13 Final Jira Dashboard and Project Report **24**
13.1 Totals . . . 25
13.2 Epics . . . 25
13.3 Work by person . . . 25

14 AI Engineering Log **25**
14.1 Method and scope . . . 26
14.2 Entries . . . 26
14.3 Human-in-the-loop statement . . . 28

A Reproducing the Evidence **28**

B Files Delivered **29**

C Rollback **29**

1 Scope, Method and Basis of the Evidence

1.1 What was run, where, and by whom

The measurements below come from a clean clone of `main` at 9f169d8 (the merge of PR #26), not from anybody's working copy. Three kinds of run are involved, and each table says which one it quotes.

- **Re-run for this document** (2 November 2026, one laptop, Docker Desktop with 8 CPUs and 4 GiB): backend, worker and frontend test suites with coverage, static analysis, dependency audits, the pilot end-to-end check, experiments E1 to E5, the load tests, the worker-kill (chaos) test, and a before/after run of the regression tests of each defect fix. The scripts are in `D3_source/scripts/` and the raw outputs in `D3_source/evidence/`.
- **GitHub Actions on 9f169d8** (run 40498905216, all five jobs green): used for the browser (e2e) suite, which was not re-run here, and as a cross-check of the numbers above. The re-run and CI agree: 318 backend tests, 85.06 % backend line coverage, the pilot check at 17 passed and 1 skipped.
- **Recorded earlier by the team** in pull-request descriptions (PR #23, PR #28, PR #30): quoted as recorded, with the PR number, and never merged silently with the re-run values.

1.2 How the measurements were isolated

The repository's own Compose file was started under a separate project name (`concordd3`) on API port 18000, with no host ports for PostgreSQL and Redis, and removed with its volumes afterwards. A developer stack on ports 5432, 6379, 8000 and 5173 kept running untouched. The tests that need a database ran in disposable PostgreSQL 16 and Redis 7 containers on a private network. The real-sandbox worker tests reused the existing local sandbox images, whose Dockerfiles are unchanged since they were built; no image tag was rebuilt.

1.3 Contents of the brief and where each item is answered

Deliverable 3 item	Section	State at the data cut
Unit, integration, system, adversarial test reports	2	591 automated tests plus 17 pilot checks, none failing
Code coverage report	3	backend 85.1 %, frontend 96.4 %, worker 73.0 %
Concurrency test report	4	17 concurrency tests, 10 and 25 editors, 32 dequeuers, 12 debug starts
Security and threat-model report	5	threat model, scans, adversarial tests, dependency audits
Performance, load and optimisation report	6	NFR-P1, P2, P3 and S3 met; E1 to E5 reported
Failure and recovery report	7	worker kill recovered in 315.5 s against a 360 s limit
Defect history in Jira	8, 13	8 bug tickets; 11 defects traced end to end
At least five defects found and fixed, with evidence	8	11 fixed, each with a before/after test run where one could be made
Costing document	9	itemised, assumptions marked
Timeline and Gantt chart	10	drawn from Jira dates
Deployment documentation	11	operator guide; production Compose validated
Final requirements-to-Jira-to-GitHub-to-testing matrix	12	26 FR and 38 NFR rows
Final Jira dashboard and project report	13	58 tasks: 42 done, 14 in progress, 2 to do
AI engineering log	14	entries for this deliverable's work

2 Unit, Integration, System and Adversarial Test Report

2.1 Results

Every automated suite passed on 9f169d8. Counts are test cases as reported by the runners (parametrised cases count once per parameter).

Suite	Runner and environment	Tests	Failed	Time	Run
Backend	pytest 8.3.3, Python 3.12, PostgreSQL 16, Redis 7	318	0	146.8 s	re-run; CI 189.0 s
Worker (real containers)	pytest, Docker daemon, three sandbox images	34	0	27.4 s	re-run
Frontend	vitest, jsdom, Node 20.20	230	0	n/a	re-run (25 files)
Browser (e2e)	Playwright in CI, Chromium	9	0	46.9 s	CI only
Pilot end-to-end check	`scripts/pilot.py check` on the Compose stack	17	0	about 167 s	re-run; CI too
Total		**591 + 17**	**0**		1 pilot step skipped (no AI key)

2.2 Tests by level

Each test file was assigned to the level that describes how it exercises the system. The assignment is a judgement of this document, made from the files' content; the full file list with its assignment follows in Section 2.4.

Level	Tests	What it exercises
Unit	323	Pure logic and client code: JWT handling, the execution state table, the priority score, DAP framing, version pruning, LLM adapters, the agent loop, and all 230 frontend component and library tests.
Integration	166	REST endpoints and the real-time gateway against PostgreSQL and Redis: sign-up, projects, invitations, files, folders, versions, snapshots, assistant, metrics, health.
System	31	Real sandbox containers (language runs, compile, cancel, debug, workspace permissions: 22 worker tests) and the browser suite (9).
Adversarial	28	Token tampering, cross-project access, path attacks, oversized input, Viewer writes (`test_adversarial_api`, `test_gateway_authorization`, `test_project_access_permissions`) and the sandbox hardening checks (`test_sandbox_security`).
Concurrency	17	Section 4.
Failure and recovery	26	Section 7.

2.3 Static analysis and builds

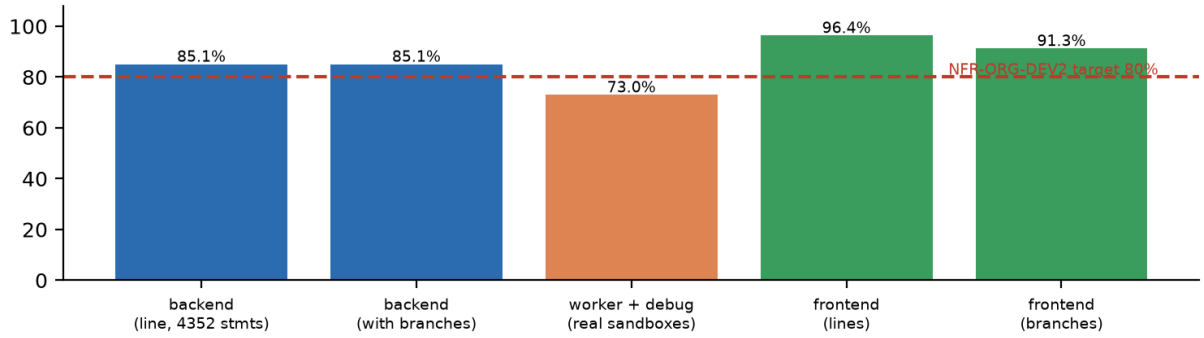
Gate	Result	Detail
ruff (E, F, W, I, B, UP, N, ASYNC, S, RUF)	pass	No findings.
mypy (`check_untyped_defs`)	pass	70 source files, no issues.
bandit `-ll`	pass	6 800 lines; 0 medium, 0 high; 3 low, all `B101` (`assert`) in `app/debug/dap.py`. One further finding is suppressed with a `#nosec` comment.
eslint	pass	0 errors, 25 warnings.
`tsc -b` and `vite build`	pass	Production bundle 722 kB gzip (Section 6).

2.4 Test inventory

Suite	File	Tests	Level	Covers
backend/tests	`test_adversarial_api`	14	Adversarial	FR-06, NFR-SEC3
backend/tests	`test_agent_runner`	24	Unit	FR-21, NFR-SEC4
backend/tests	`test_assistant_api`	22	Integration	FR-21, FR-22
backend/tests	`test_auth`	8	Integration	FR-01
backend/tests	`test_awareness`	16	Unit	FR-09
backend/tests	`test_breakpoints`	4	Integration	FR-17
backend/tests	`test_checkpoint`	1	Integration	NFR-D2
backend/tests	`test_concurrent_edits`	1	Concurrency	FR-08, NFR-D3
backend/tests	`test_dap_client`	4	Unit	FR-17
backend/tests	`test_debug`	6	Integration	FR-17, FR-18
backend/tests	`test_debug_session`	9	Integration	FR-18, NFR-P3
backend/tests	`test_debug_single_slot`	1	Concurrency	FR-18
backend/tests	`test_execution_delivery`	6	Failure	FR-14
backend/tests	`test_execution_events`	13	Integration	FR-10, NFR-ORG-OP3
backend/tests	`test_execution_state_machine`	6	Unit	FR-10
backend/tests	`test_failure_recovery`	3	Failure	FR-16, NFR-D2
backend/tests	`test_file_changes`	17	Integration	FR-25
backend/tests	`test_files`	14	Integration	FR-07
backend/tests	`test_folders`	14	Integration	FR-07
backend/tests	`test_gateway_authorization`	1	Adversarial	FR-06
backend/tests	`test_gateway_convergence`	2	Concurrency	FR-08, NFR-D3
backend/tests	`test_health`	8	Integration	NFR-ORG-OP1
backend/tests	`test_invitations`	6	Integration	FR-03
backend/tests	`test_join_requests`	9	Integration	FR-04
backend/tests	`test_llm_adapters`	15	Unit	FR-20
backend/tests	`test_llm_manager`	8	Unit	FR-20, FR-24
backend/tests	`test_members`	8	Integration	FR-05
backend/tests	`test_metrics`	3	Integration	FR-23
backend/tests	`test_migrations_enums`	2	Integration	NFR-ORG-ENV3
backend/tests	`test_orphan_sweep`	17	Failure	FR-16, NFR-D4
backend/tests	`test_permissions`	1	Integration	FR-06
backend/tests	`test_priority_queue`	9	Concurrency	FR-15, NFR-D5
backend/tests	`test_project_access_permissions`	1	Adversarial	FR-06
backend/tests	`test_queue_atomic_claims`	4	Concurrency	FR-14, NFR-D5
backend/tests	`test_realtime_gateway`	5	Integration	FR-08, FR-09
backend/tests	`test_run_input`	4	Unit	FR-11
backend/tests	`test_sandbox_workspace`	3	Unit	NFR-SEC1
backend/tests	`test_search`	5	Integration	FR-26
backend/tests	`test_security`	8	Unit	NFR-SEC2, NFR-EXT-REG2
backend/tests	`test_snapshots`	3	Integration	FR-19
backend/tests	`test_version_pruning`	5	Unit	NFR-S4
backend/tests	`test_versions`	8	Integration	FR-25
worker/tests	`test_cancel_running`	2	System	FR-13 (real containers)
worker/tests	`test_debug_session`	3	System	FR-17 (real containers)
worker/tests	`test_sandbox_security`	12	Adversarial	NFR-SEC1, NFR-EXT-LEG-SEC1 (real containers)
worker/tests	`test_worker_execution`	4	System	FR-10, FR-11 (real containers)
worker/tests	`test_worker_workspace`	13	System	NFR-SEC1 (real containers)
frontend	`api`	35	Unit	client components and libraries
frontend	`FileTree`	9	Unit	client components and libraries
frontend	`DashboardPage`	10	Unit	client components and libraries
frontend	`AssistantPanel`	18	Unit	client components and libraries
frontend	`ws`	20	Unit	client components and libraries
frontend	`ConsolePanel`	9	Unit	client components and libraries
frontend	`debug`	5	Unit	client components and libraries
frontend	`DebugPanel`	6	Unit	client components and libraries
frontend	`HistoryModal`	6	Unit	client components and libraries
frontend	`errors`	7	Unit	client components and libraries
frontend	`Editor`	24	Unit	client components and libraries
frontend	`LoginPage`	5	Unit	client components and libraries

Suite	File	Tests	Level	Covers
frontend	`SnapshotsPanel`	6	Unit	client components and libraries
frontend	`SignupPage`	3	Unit	client components and libraries
frontend	`Markdown`	6	Unit	client components and libraries
frontend	`QuickOpen`	2	Unit	client components and libraries
frontend	`EditorTabs`	2	Unit	client components and libraries
frontend	`SearchPanel`	4	Unit	client components and libraries
frontend	`RecentChanges`	4	Unit	client components and libraries
frontend	`SessionPage`	35	Unit	client components and libraries
frontend	`ErrorBoundary`	3	Unit	client components and libraries
frontend	`Toasts`	2	Unit	client components and libraries
frontend	`tabs`	3	Unit	client components and libraries
frontend	`fuzzy`	4	Unit	client components and libraries
frontend	`languages`	2	Unit	client components and libraries
e2e (browser)	`test_navigation`	2	System	NFR-U1, NFR-U4
e2e (browser)	`test_collaboration`	2	System	FR-08, FR-09
e2e (browser)	`test_execution`	4	System	FR-10, FR-12, FR-13, NFR-SEC3
e2e (browser)	`test_debugger`	1	System	FR-17, FR-18

3 Code Coverage Report



Suite	Lines or statements	Branches	Measured over
Backend (`pytest --cov=app --cov-branch`)	85.06 % (3 757 of 4 352)	79.2 % (741 of 936)	Python 3.12; same figure in CI.
Worker + debugger + sandbox module	72.99 % (992 statements)	included	Real containers; modules `worker.app`, `app.debug`, `app.services.sandbox`.
Frontend (vitest, v8)	96.36 %	91.26 %	25 files; functions 85.07 %.

Reading the numbers. The backend figure is above the 80 % target of NFR-ORG-DEV2. The backend run alone shows the container-dependent modules (`app/services/sandbox.py`, `app/debug/docker_adapter.py`) at low line coverage because those lines run only with a real container; the worker suite covers them (64 % and 83 %).

The HTML coverage reports are kept in the evidence folder (`backend_htmlcov/`, `backend_coverage.xml`, `worker_coverage.json`, `frontend_coverage/`); CI uploads the same artifacts on every run.

4 Concurrency Test Report

Concord has two shared things that several actors can touch at once: a collaborative document (many editors, no lock) and the execution slots (two workers, many waiting runs). Deliverable 1 stated one correctness property for each: replicas converge under any ordering (NFR-D3), and the N running executions are always the N highest-priority waiting ones with no slot assigned twice (NFR-D5, FR-15). This section tests both,

plus the one-debug-session-per-project rule (FR-18).

4.1 Tests and outcomes

Test	Scenario	Property	Result
`test_gateway_convergence` [10] and [25]	10, then 25 clients type into one file through the real WebSocket gateway, in-process	identical final text, no lost or duplicated edit (NFR-D3)	2.8 s, 6.9 s pass
`test_concurrent_edits_converge`	updates applied in different orders to separate replicas	same state under any ordering (NFR-D3)	pass
`test_concurrent_dequeuers_never_claim_the_same_execution` [2], [8], [32]	2, 8 and 32 dequeuers race for the same waiting set	each execution claimed by exactly one dequeuer (NFR-D5)	pass
`test_a_single_waiting_execution_goes_to_exactly_one_of_many_dequeuers`	one entry, many claimants	exactly one winner	pass
`test_priority_queue` (9 tests)	role weight, ageing, compile penalty, debug bonus	highest score first; ageing always overtakes the largest role gap (no starvation)	pass
`test_concurrent_debug_starts_in_one_project_give_exactly_one_session`	12 concurrent debug starts in one project	one succeeds, 11 refused with 409 (FR-18)	pass

4.2 Live concurrency measurements

Edits on the wire (load test). Ten, then twenty-five WebSocket clients edited one file for about 45 s while five runs were submitted together, three rounds. Deliveries were counted, not sampled.

Editors	Edits sent	Delivered / expected	p50 ms	p99 ms	max ms
10	2 000	18 000 / 18 000	4.6	11.3	21.8
25	5 000	120 000 / 120 000	9.7	50.1	92.8

No edit was lost or duplicated at either size.

Slot contention (experiment E3). Both workers were occupied, then an Editor (bob), a Debugger (dave) and the Owner (alice) each submitted a run within 1.1 ms. The runs started in the order **owner, debugger, editor**, after 240, 272 and 484 ms. That is the order the priority score predicts (Owner above Debugger above Editor), not arrival order. The two blocking runs held both slots until a slot freed, and each freed slot went to exactly one waiting run.

5 Security and Threat-Model Report

5.1 Assets, actors and trust boundaries

What is worth protecting. (1) The host that runs student code: the worker mounts the Docker socket, which is root-equivalent on that machine (ADR-004). (2) Project source and execution output, which belong to a project's members only. (3) Account credentials and the JWT signing key. (4) Project-owner LLM API keys, stored encrypted. (5) Availability of the two execution slots, which are finite (Section 4).

Who can attack. A *Guest* with no account; a signed-in *non-member*; a *Viewer* trying to exceed read-only; an *Editor* whose code is hostile; a project member abusing the *AI assistant*; an outside party with network access to the exposed port.

Trust boundaries. (B1) browser to Caddy to API: TLS, JWT. (B2) API to PostgreSQL and Redis: private Compose network, no published ports in production. (B3) worker to Docker daemon: socket. (B4) **sandbox**

container to everything else: the principal boundary, because the code inside is hostile by design. (B5) API to the external LLM provider: HTTPS, key per project.

5.2 Threat model

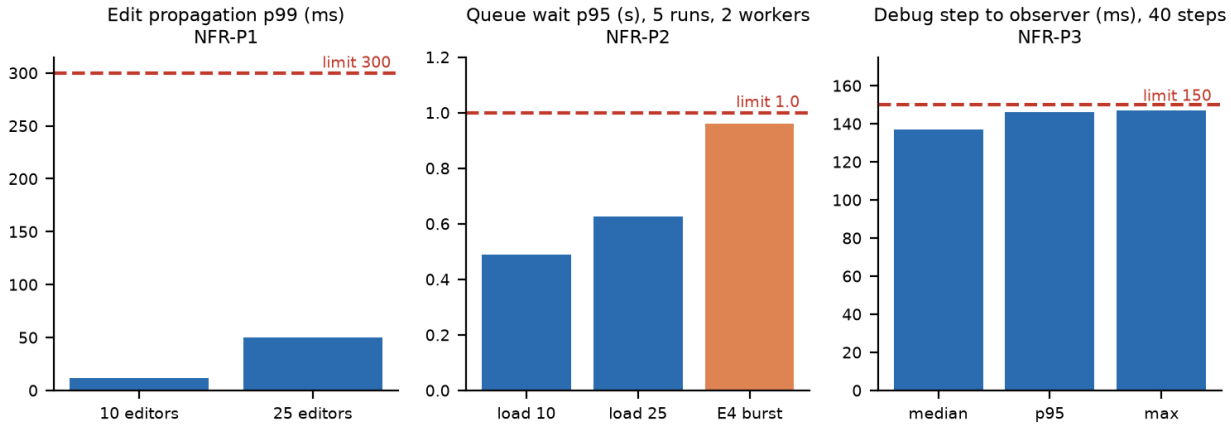
STRIDE applied to each boundary. "Control" names the mechanism as built; "Evidence" names the test or measurement that supports it; "Residual" is what the evidence does not cover.

ID	Threat	Control as built	Evidence
T1 S	Forged or tampered JWT; `alg=none`; token signed with another key; expired token; token of a deleted user	HS256 only, algorithm fixed by the server, 24 h expiry, user looked up on each request	6 token tests in `test_adversarial_api`, 8 in `test_security`: all rejected
T2 S	Password guessing	bcrypt hashes; 5 failures lock the account name for 900 s (CONCORD-66, PR #24)	`test_sixth_wrong_password_is_rate_limited`; before/after run in Section 8
T3 T	Path traversal and absolute paths in file and folder names (CONCORD-65)	Shared path validator on create, rename, move	`test_traversal_paths_are_rejected_on_file_and_folder_writes`, 28 file and folder tests
T4 T	Viewer writes through REST or WebSocket	Role guard on every REST route and every inbound edit message	`test_viewer_websocket_edit_is_rejected`, `test_viewer_cannot_call_project_write_routes`, browser test `test_a_viewer_cannot_edit_or_run`
T5 I	Reading another project's files, executions or WebSocket rooms	Project membership checked at REST and at the WebSocket handshake	`test_non_member_cannot_access_protected_project_routes`, `test_websocket_cannot_open_file_from_another_project`, `test_a_file_of_another_project_is_refused`
T6 I	**Execution metrics readable without logging in** (CONCORD-63)	none: the metrics routes take no user dependency	**Confirmed live on 2 Nov: `GET /metrics/executions/summary` with no token returned HTTP 200** and the counts of all runs
T7 I	Provider API key exposure	Fernet encryption at rest; only the Owner may set it	`test_llm_manager`, `test_assistant_api`
T8 E	**Escape from the sandbox**: capabilities, new privileges, fork bomb, memory exhaustion, network, host file system	network disabled; `cap_drop=ALL`; `no-new-privileges`; non-root uid 10001; read-only root file system; 128 MB; 0.5 CPU; 64 processes; workspace mounted read-only for run, read-write only for the compile step	12 tests in `test_sandbox_security` on real containers: 7 passed and 5 failed *before* PR #29 (CONCORD-62), 12 passed after; pilot-check steps "Sandbox has no network" and "Sandbox stops infinite loops"
T9 E	Viewer or Editor reaching Debugger or Owner actions	four-level hierarchy; debug commands need Debugger or Owner	pilot check "Debug together" (bob, an Editor, could not step), role tests in `test_debug` and `test_permissions`
T10 D	Run flood starving others	finite slots; priority with ageing (no starvation); 64 KiB stdin cap; 10 s run limit; 5 min sweeper	E3, E4, `test_standard_input_larger_than_64_kib_is_rejected`
T11 D	AI request flood and cost	10 requests per 60 s per project in Redis	`test_llm_manager`
T12 R	A member denies an action	append-only execution event log; AI requests and accepted proposals recorded with the accepting member	`test_execution_events` (13), `test_assistant_api`
T13 T	AI agent changes code without approval, or abuses tools, or follows injected text	agent has no write tool; `propose_edit` only records a proposal; a human accepts through the normal edit path; tools scoped to the project; 6-step cap	`test_agent_runner` (24), `AssistantPanel.test.tsx`

5.3 Scans

Scan	Result	Findings
bandit (6 800 lines)	0 medium, 0 high	3 low: `assert` in `debug/dap.py`.
pip-audit (48 pinned packages)	0 advisories	PR #28 replaced `python-jose` by PyJWT to remove advisories; the audit is clean on 9f169d8.
npm audit, production dependencies	2 moderate	`react-router` (and `react-router-dom`): open redirect through a backslash in `<Link>`; a fixed version exists.
npm audit, all dependencies	3 critical, 3 high, 19 moderate	Development tooling only (vitest, vite, esbuild, `@vitest/coverage-v8`, eslint plugins); not in the production bundle. Still worth upgrading: the vitest and vite advisories concern a developer's own machine while `npm run dev` or the vitest UI is listening.
Secrets in the repository	no finding	`.env` and `.env.production` are ignored by Git; `.env.example` holds placeholders only.

6 Performance, Load Test and Optimisation Report



6.1 Requirements against measurements

Req.	Target (Deliverable 1)	Measured here	Earlier team runs	Verdict
NFR-P1	99 % of edits reach all clients in under 300 ms, 10 editors	p99 11.3 ms (10), 50.1 ms (25); max 92.8 ms; 100 % delivered	p99 23.9 to 198.7 ms, five runs (PR #30)	met
NFR-P2	QUEUED to STARTING under 1.0 s at p95, up to 5 simultaneous runs	p95 0.488 s and 0.625 s (load runs, 15 samples each); 0.959 s in E4 (50 samples)	p95 1.23 to 1.63 s in all five runs (PR #30)	met
NFR-P3	debug step visible to others in under 150 ms	40 steps: median 136.8, p95 146.3, max 146.8 ms; none above 150	median 142.97, p95 162.13 ms over 120 steps (PR #28)	met
NFR-S3	client bundle under 2.5 MB compressed	722 kB gzip (JS 707.6, CSS 14.1, HTML 0.5)	none	met
NFR-S1, S2	128 MB and 0.5 CPU per sandbox	enforced by the container options, which the worker tests inspect; the pilot check stops an infinite loop	none	enforced

6.2 Experiments E1 to E5

These are the repository's own reproducible scripts (`scripts/experiments`, CONCORD-67), run once each against the isolated stack with two workers.

ID	What it measures	Result
E1	Status arrival times for one Python and one C++ run	Python: queued 29 ms, starting 61 ms, running 81 ms, completed 316 ms after submit. C++: queued 28 ms, starting 50 ms, compiling 79 ms, running 617 ms, completed 773 ms. The compile step costs about 540 ms.
E2	Cancel a run 1.5 s after it starts; time until its container is gone	HTTP 200, final status `cancelled`, container absent within **106 ms** of the cancel call (poll interval 100 ms). Fixes CONCORD-60.
E3	Start order when an Editor, Debugger and Owner submit while both workers are busy	Owner, Debugger, Editor, after 240, 272 and 484 ms (Section 4).
E4	Queue wait for five simultaneous runs, 10 rounds	50 samples: median 424 ms, p95 959 ms, max 983 ms.
E5	Command-to-observer latency over 40 debugger `next` steps	median 136.8 ms, p95 146.3 ms, max 146.8 ms.

6.3 Optimisation results

Change	Cause found	Before	After
CONCORD-71, PR #28: publish the paused location before loading frame variables	Command to paused-event publication took 189 ms because stack trace, scopes and variables (about 46 ms each) were fetched first	median 201.2 ms, p95 222.8 ms (120 steps)	median 143.0 ms, p95 162.1 ms (PR #28); **136.8 / 146.3 ms in this re-run** (40 steps)
CONCORD-57, PR #30: load-test script	No repeatable way to check NFR-P1 and NFR-P2 together	none	Script reports p50, p99, deliveries and queue wait; exits 1 on a miss (it did on all five earlier runs).
Compile cost of C++ (observed, not changed)	Compiling takes about 540 ms of a 773 ms run (E1)	none	Left as is. The priority score already weights C++ down under contention.

The debug-step gain is 58 ms at the median (29 %) in the team's measurement. It was made by changing *when* data is published, not by making the debugger faster: step stops now send the location first and variables afterwards in a `debug_frame` event, so a client briefly sees the new line before the variables fill in.

6.4 Resource use

Idle memory of the running stack (read from `docker stats` on the development stack, no load): backend 111 MiB, each of two workers 60 MiB, PostgreSQL 46 MiB, Redis 11 MiB; the development frontend server 223 MiB, which the production Caddy image replaces. Each sandbox adds at most 128 MiB while it runs. Image sizes: backend and worker 699 MB each, frontend 645 MB (development image), sandboxes 256 MB (Python), 194 MB (Node), 486 MB (C++).

7 Failure and Recovery Test Report

Deliverable 1 named four failures the design must survive: a worker dying mid-run (FR-16, NFR-D4), memory exhaustion and timeouts (NFR-S1, FR-10), cancellation (FR-13) and contention (Section 4). Deliverable 2 added Redis and backend restarts as risks created by the build. Each is tested below.

7.1 Scenarios, mechanisms and results

Failure injected	Mechanism that handles it	Test or measurement	Result
Worker process killed (SIGKILL) while it holds a run	Background sweeper marks runs stuck for over 5 min as FAILED, appends the event in the same transaction, signals cancel, removes the container	`scripts/chaos.py`: claiming worker killed, then restarted; `test_stuck_run_is_swept_logged_and_its_containers_removed`	FAILED after **315.5 s** (limit 360 s); sweeper event present; **0 containers left**
Docker briefly unavailable during cleanup	Sweep retries container removal on later passes after the row is already FAILED (review finding, PR #23)	`test_sweep_retries_failed_container_removal_after_restart`, `test_sweep_retries_terminal_cleanup_after_container_listing_recovers`	pass
Redis unreachable when a run is submitted	Run is recorded as FAILED, the debug claim is released, the client gets 503	`test_redis_outage_returns_503_and_records_failed_run`, `test_queue_failure_persists_failed_run_and_releases_debug_claim` (4 parametrisations)	pass
Redis outage and the cleanup also fails	The persisted failure is not hidden by the second error	`test_queue_failure_cleanup_outage_does_not_hide_persisted_failure`	pass
Delivery acknowledgement lost after a worker already claimed the run	The run is not failed or restarted by the retry	`test_lost_delivery_ack_does_not_fail_a_run_already_claimed_by_worker`; worker test: non-queued runs are never restarted (4 states)	pass
User cancels in each phase	Status transition plus container stop	`test_cancel_in_each_phase_leaves_no_container` (C++ compiling, C++ running), `test_cancel_removes_running_container_within_one_second`; E2	container gone in 106 ms (E2)
Cancel races with QUEUED to STARTING	Cancel retries when the worker advances the state (review finding, PR #23)	`test_cancel_retries_when_worker_advances_to_another_active_state`, `...after_more_than_one_worker_advance`	pass
Runaway program	10 s timeout; memory, CPU and PID limits	pilot step "Sandbox stops infinite loops" (status `timeout`); worker timeout test	pass
Active debug session left idle or running	600 s hard limit, 180 s idle limit, a separate recovery grace; the sweeper must not kill a live session	`test_sweep_preserves_live_debug_session_beyond_five_minutes`, `...respects_configured_debug_timeout`, `...recovers_debug_run_after_timeout_and_recovery_grace`	pass
Backend restarted	Last checkpoint restored from PostgreSQL	`test_backend_restart_restores_last_checkpoint`	pass for the restored text
Illegal status change (for example COMPLETED to RUNNING)	Allowed-transitions table; status and event commit or roll back together	13 tests in `test_execution_events`, 6 in `test_execution_state_machine`; before/after in Section 8	pass

7.2 The worker-kill run in detail

The chaos script was pointed at the isolated Compose project, replaced a disposable file with a long-running program, submitted it, identified the worker that had claimed it, killed that worker with SIGKILL, restarted it, and waited. Its report:

```
{"execution_id": 99, "status": "failed", "recovery_seconds": 315.522, "sweeper_event": true,
                    "containers_remaining": [], "passed": true}
```

The recovery time is below the 360 s bound of NFR-D4 (5 min threshold plus one 60 s sweep). The run recorded in PR #23 took 338.2 s and this one took 315.5 s; both fall inside the 60 s sweep interval after the 300 s threshold, which is what a sweeper that runs every 60 s should produce. Two runs on two machines are a pair of samples, not a distribution.

8 Defect History and Evidence of Fixes

8.1 How defects were found

Defects entered the backlog by four routes, and this section keeps them apart because they say different things about the process: (a) failures seen when running the system for Deliverable 2 (CONCORD-59); (b) bug tickets written from a review of the code against the requirements (CONCORD-60 to CONCORD-66, label `bug`); (c) problems a reviewer found in a pull request before merging (PR #23, PR #27); (d) performance and dependency issues found by measurement (CONCORD-71, CONCORD-72).

8.2 Defects found and fixed

No.	Jira	Fix	Defect	Cause and fix	Evidence
D1	CONCORD-59 (route a)	PR #20	Application did not start from a fresh clone of `main`	`main.py`, the client entry point, `conftest.py` and `ci.yml` were missing from `main`, and `pilot.sh` failed on macOS (BSD `sed -i`); the repair PR adds them (2 594 lines)	Regression files for PR #20: 21 errors on the code before, 21 pass after (**note**: before-errors are set-up failures, because the fix introduced the fixtures; the run shows the tests cannot run on the old tree, not a behavioural assertion)
D2	CONCORD-60 (b)	PR #23	Cancelling a running execution did not stop its sandbox container	The cancel signal cancelled only the asyncio task that waits on a thread; the thread and its container ran on until the 10 s limit (8.5 s later in E2 of Deliverable 2). The worker now kills the container	Worker tests: **4 failed, 2 passed before; 6 passed after**; E2 shows the container gone within 106 ms
D3	CONCORD-61 (b)	PR #23	Execution status changes were not checked against the allowed-transitions table	The allowed-transitions table was used only by its own test; worker, cancel endpoint and sweeper wrote statuses directly, so a late worker update could move CANCELLED back to RUNNING. `execution_state.py` is now the single write path, with the event in the same transaction	Backend tests for PR #23: collection error before (module absent), **39 passed after**
D4	CONCORD-62 (b)	PR #29	Sandbox containers kept Linux capabilities and could gain privileges	Run, build and debug containers started without `cap_drop` or `security_opt`; now `cap_drop=ALL` and `no-new-privileges` on all three	`test_sandbox_security`: **5 failed, 7 passed before; 12 passed after**. Failing assertions: `NoNewPrivs` 0 where 1 is required; `cap_drop` absent. (`CapEff` was already 0 for the non-root user, so the privilege-gain flag and the explicit drop were the changes needed.)
D5	CONCORD-65 (b)	PR #24	File and folder paths accepted `..`, absolute paths, empty segments, backslashes, control characters	The schemas checked only length (1 to 512), so `../../etc/passwd` and `a\b.py` were stored, shown in the tree and copied into snapshots (the worker refused them only at run time). A shared validator now rejects them on create and rename	`test_files`, `test_folders`: **14 failed, 14 passed before; 28 passed after**
D6	CONCORD-66 (b)	PR #24	Login had no limit on repeated failures	`POST /auth/login` accepted unlimited wrong passwords. Redis counters now lock a name after 5 failures for 900 s	Backend tests of PR #24: import error before (`login_limiter` absent), **50 passed after**, including `test_sixth_wrong_password_is_rate_limited`
D7	(c)	PR #23	Cancel returned 200 without cancelling when the worker moved QUEUED to STARTING a moment earlier (race)	Cancel retries when the state advances	`test_cancel_retries_when_worker_advances_to_another_active_state` and `...after_more_than_one_worker_advance` pass
D8	(c)	PR #23	After a transient Docker error the row was FAILED but the container kept running; later sweeps skipped it	Sweep retries terminal cleanup	`test_sweep_retries_failed_container_removal_after_restart` and `...terminal_cleanup_after_container_listing_recovers` pass

No.	Jira	Fix	Defect	Cause and fix	Evidence
D9	(c)	PR #27	Experiment E3 could report `complete=true` after its blockers had timed out, so a contender started without contention	Blockers are re-checked after the burst; the run is marked incomplete	Reviewer reproduced it with scripted events. `scripts/experiments/tests/test_contention.py`: 6 tests, 6 passed in this re-run
D10	CONCORD-71 (d)	PR #28	Debug step took 201 ms at the median to reach other members (target 150)	Paused location published before variables load	Median 201.2 to 143.0 ms (team); 136.8 ms in this re-run (Section 6)
D11	CONCORD-72 (d)	PR #28	Python dependencies carried known advisories, and the JWT library `python-jose` was among them	Packages updated, `python-jose` replaced by PyJWT, eight JWT regression cases	pip-audit: **0 advisories in 48 packages** on 9f169d8

How the before/after runs were made. For each fix, the checkout of the PR's base commit received only the *test files* of the PR's merge commit, the tests were run, then the same files were run on the merge commit. Test files that import a module the fix creates cannot even be collected on the old code; that is why D3 and D6 show an import error instead of counted failures, and why D5 was repeated on the two files that run on both trees. Raw output: `evidence/regression_before_after.log`; script `D3_source/scripts/run_regression_before_after.sh`. Dependencies were installed per commit, because the base commits still used `python-jose`; the first attempt without that failed to import `jose`, which is why the log shows one corrected run.

8.3 Bug and defect history in Jira

Jira holds eight tickets labelled `bug` (CONCORD-59 to CONCORD-66) and twenty-two labelled `deliverable-3`. The state of the bug tickets on 2 November 2026:

Key	Summary	Assignee	Jira status	Fix merged
CONCORD-59	Fixing bugs: application does not start from a fresh clone of main	Aamir Ahmad	Done	PR #20
CONCORD-60	Cancelling a running execution does not stop its sandbox container	Aryamann Srivastava	In Progress	PR #23
CONCORD-61	Execution status changes are not checked against the allowed-transitions table	Aryamann Srivastava	In Progress	PR #23
CONCORD-62	Sandbox containers keep Linux capabilities and can gain new privileges	Aryamann Srivastava	In Progress	PR #29
CONCORD-63	Metrics endpoints are readable without logging in and show every project's runs	Sanchit Arora	In Progress	none
CONCORD-64	AI tool calls are not stored in the request audit record	Sanchit Arora	In Progress	none
CONCORD-65	File and folder paths accept `..`, absolute paths and empty segments	Pallav Rastogi	In Progress	PR #24
CONCORD-66	Login does not limit repeated failed attempts	Pallav Rastogi	In Progress	PR #24

9 Costing Document

Reading this section. Rows marked **S** use a price found on the web on 2 November 2026 (third-party price aggregators; the vendors' own pages were not opened, so check before budgeting). Rows marked **A** are assumptions of this document, stated so the reader can change them. Rows marked **M** are measured on this project. No money has been spent on infrastructure: no production host exists (Section 11), so everything below is a plan priced from measurements, not an invoice.

9.1 Sizing from measurements

The idle stack uses about 310 MiB (backend 111, two workers 60 each, PostgreSQL 46, Redis 11, plus an estimated 20 for Caddy) **M**. Each sandbox is capped at 128 MiB and 0.5 CPU **M**. The load tests ran 25 editors and five simultaneous runs on two workers inside a Docker VM of 8 CPUs and 4 GiB **M**. A host with 2 vCPU and 8 GiB therefore has room for the stack, the sandboxes and the operating system with a wide margin; whether a 1 GiB free-tier instance would hold it was not tested, and 310 MiB idle plus two sandboxes plus the Docker daemon leaves little room.

9.2 Monthly operating cost

Item	Basis	USD / month	Kind
Compute: one AWS EC2 `t3.large` (2 vCPU, 8 GiB), us-east-1	\$0.0832 per hour × 730 h	60.74	S
Block storage, 50 GB gp3	\$0.08 per GB-month (list price)	4.00	A
Container registry (ECR), about 2.1 GB of images	\$0.10 per GB-month; sizes from `docker images`	0.21	A, M
Domain name	about \$12 per year	1.00	A
Off-host database backups, about 1 GB	\$0.023 per GB-month (object storage)	0.02	A
Outbound traffic	first 100 GB per month free	0.00	A
CI (GitHub Actions), public repository	no charge for public repositories	0.00	A
Issue tracking (Jira, 5 users)	free plan	0.00	A
TLS certificates (Caddy, automatic)	no charge	0.00	n/a
Infrastructure subtotal		**65.97**	
LLM requests (Section below)	500 requests per month	2.70	A
Total, one month		**68.67**	
Total, a four-month semester	4 × 68.67	**274.68**	

9.3 LLM cost per request and its ceiling

The default provider is Mistral Codestral. Its list price found on 2 Nov is \$0.30 per million input tokens and \$0.90 per million output tokens (one aggregator lists other values; the Mistral price page should be checked) **S**. An agent request runs at most six steps. Assuming 12 000 input tokens and 2 000 output tokens in total per request **A**:

$$12\,000 \times 0.30/10^6 + 2\,000 \times 0.90/10^6 = 0.0036 + 0.0018 = \$0.0054 \text{ per request.}$$

The project limit of 10 requests per 60 s (FR-24) caps the damage of a flood at \$0.054 per minute, \$3.24 per hour and \$77.76 per day per project. The monthly line above (500 requests, \$2.70) is an assumption about a laboratory term; the real figure can be read from the stored token counts (NFR-EXT-LEG-ACC2), which the metrics do not yet aggregate. A project may supply its own key, in which case this cost is the owner's.

9.4 Cost per run

A 0.5 s Python run on 0.5 CPU uses about $0.5 \times 0.5\,\text{s} \times$ (\$0.0832 / 2 vCPU / 3600 s) = \$0.000003 of compute. The cost of a run is negligible; the cost driver is the always-on host.

9.5 Scenarios

Scenario	Assumption	USD / month	Kind
Demonstration, as measured	one `t3.large`, two workers	68.67	S, A
Meet NFR-P2 with margin	three workers (Section 6) on a `t3.xlarge` (4 vCPU, 16 GiB) at $0.1664 per hour	129.40	A
Stopped outside laboratory hours	compute running 60 h per week (about 261 h per month) instead of 730, rest unchanged	29.62	A

The second row is a doubling of the compute price (the `t3` family prices linearly with size) and has not been tested; the third assumes the host is stopped and started on a schedule, which the repository does not do today.

9.6 Development effort

Jira holds 58 tasks with 248 story points; 195 points are in Done tasks. With an assumed 2.5 hours per point **A** and a notional student rate of $15 per hour **A** the plan is 620 hours ($9 300) and the completed work 487.5 hours ($7 313). This is an equivalent value, not money paid. Spread over five people and the six weeks from 26 September to 6 November, 620 hours is about 21 hours per person per week. AI coding assistants were used by members under their own subscriptions (Section 14); those costs are not charged to the project and are not included.

10 Timeline and Gantt Chart



Sources. Bars for the two sprints use the board's sprint dates (Sprint 1, 8 to 15 October, state "future" in Jira; Sprint 2, 19 October to 6 November, active). Task bars run from the Jira creation date to the Jira resolution date; open tasks are hatched out to the sprint end because no other date exists. The Deliverable 1 sprint window is the plan of Deliverable 1, Section 11. Deliverable 2 was submitted on 16 October.

10.1 Plan against what happened

Item	Deliverable 1 plan	Observed
Sprints	four two-week sprints, 26 Sep to 21 Nov	two sprints on the board; Sprint 1 never moved from "future" to "closed" in Jira, so its 35 tasks carry no completed-sprint record
Execution and debugging	Sprint 3, 24 Oct to 7 Nov	merged 15 October to 25 October (PR #10, PR #12, PR #15, PR #23)
Priority scheduler (CONCORD-42, CONCORD-43)	Sprint 3 milestone	merged before 15 October (Deliverable 2); concurrency evidence added 23 October (PR #21)
AI assistant	Sprint 4, 7 to 21 Nov	merged 15 October (PR #17, PR #18, PR #19)
Load testing	Sprint 4	script merged 25 October (PR #30); run here on 2 November
Deliverable 3 package	6 November	this document, with CONCORD-77 (integration, release) and CONCORD-78 (dashboard, closeout) as its closing tasks

The implementation work ran ahead of the Deliverable 1 calendar, and the testing and hardening work was spread over the last three weeks. Pull requests merged per ISO week: 20 in week 42 (12 to 18 Oct), 9 in week 43, 1 in week 44. **Commit timestamps are not used as evidence of when effort was spent anywhere in this document;** the dates quoted are Jira creation and resolution dates and pull-request merge dates.

11 Deployment Documentation

This section is the operator's guide, written to match the repository's `docs/deployment.md`, `docker-compose.prod.yml`, `deploy/` and `.github/workflows/deploy.yml` at 9f169d8. The repository file is the working copy that is kept up to date with the code; if the two ever differ, the repository wins.

11.1 Architecture of the production deployment

One Linux Docker host runs six containers on a private Compose network. **Caddy** is the only service with published ports (80 and 443): it terminates TLS with automatic certificates and forwards requests to the **frontend** container (the built static client, internal port 80) and to the backend for the API and WebSocket paths. **backend** (FastAPI) runs `alembic upgrade head` before it serves. **worker** starts after the backend is healthy, mounts the host Docker socket and starts one sandbox container per run. **postgres** and **redis** have named volumes and no published ports; Redis uses append-only persistence. Every service restarts `always`; PostgreSQL, Redis and the backend have health checks, and two worker replicas run.

11.2 Prerequisites

Ubuntu 22.04 or newer; Git, curl, OpenSSL, Docker Engine and the Compose plugin; a DNS name pointing at the host; inbound TCP 80 and 443 (SSH restricted to administrators); disk for three sandbox images (about 0.94 GB), the backend and worker images (about 1.4 GB together) and backups. **Use a dedicated host**: the worker's Docker socket is root-equivalent on that machine (Section 5, T8).

11.3 Configuration

Keep `.env` and `.env.production` on the server only, mode 600; both are Git-ignored. Required values: `APP_ENV=prod`; a fresh `SECRET_KEY` and `LLM_ENCRYPTION_KEY` (`openssl rand -hex 32` each; keep the encryption key across deployments or stored provider keys become unreadable); a strong `POSTGRES_PASSWORD` matched in `DATABASE_URL`; `CONCORD_DOMAIN` and `CORS_ORIGINS=https://<domain>`; optionally `MISTRAL_API_KEY`. `.env.production` pins immutable image references (`BACKEND_IMAGE`, `WORKER_IMAGE`, `FRONTEND_IMAGE`) built from one tested commit. The frontend bundle is built with `VITE_API_URL=https://<domain>/api` and `VITE_WS_URL=wss://<domain>`; changing runtime variables does not change a built bundle.

11.4 First deployment and updates

```
./deploy/setup.sh                        # fresh Ubuntu host: Docker, local config, 3 sandbox images
./deploy/compose.sh config --quiet       # validates the file and variables
./deploy/compose.sh pull
./deploy/compose.sh up -d --remove-orphans --wait --wait-timeout 180
./deploy/compose.sh ps
curl -fsS https://<domain>/healthz
```

Then sign in and run a small Python program: a health check does not prove that workers can launch a sandbox. For an update, take a backup, note the current SHA and image pins, check out the tested revision and repeat. Never run `down -v` on a production database.

11.5 Automated deployment

The `deploy` workflow starts after `ci` succeeds for a push to `main`, builds the exact tested SHA, pushes to ECR (region us-east-1) and updates the server over SSH. It needs these repository secrets: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `ECR_REPO_BACKEND`, `ECR_REPO_WORKER`, `ECR_REPO_FRONTEND`, `CONCORD_DOMAIN`, `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY`, `DEPLOY_HOST_FINGERPRINT`. It keeps the previous image pins in `.env.production.bak` and fails the run if readiness is not reached. It creates no infrastructure.

11.6 Health, logs and monitoring

`GET /healthz` needs no token and returns `database`, `redis` and `queue_depth`: HTTP 200 and `status=ok` when both respond; HTTP 503 when the database fails; HTTP 200 with `status=degraded` when only Redis fails (queue depth unknown). Each check has a two-second deadline. The pilot check observed `{status: ok, database: ok, redis: ok, queue_depth: 0}`. Logs: `./deploy/compose.sh logs --tail 100 backend worker caddy`.

11.7 Backup and restore

```
mkdir -p backups && chmod 700 backups
./deploy/compose.sh exec -T postgres sh -c \
  'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc' > backups/concord.dump
# restore, in a maintenance window, with writers stopped:
./deploy/compose.sh stop backend worker
./deploy/compose.sh exec -T postgres sh -c \
  'pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists --exit-on-error' < backups/concord.dump
./deploy/compose.sh up -d --wait --wait-timeout 180
```

Copy backups off the host and test a restore into a separate database before relying on one. A database backup does not contain the Redis queue or live debug state: finish or stop outstanding runs before maintenance. Keep the encryption key with the backups.

11.8 Rollback

Record the deployed SHA and the previous image pins before updating. On failure: stop writes, read the logs, check out the previous revision, restore its pins in `.env.production`, pull and `up -d --wait`. Keep the current `.env`. Do not roll code back across an incompatible schema change: review the migration and either apply its reviewed downgrade or restore the backup first. After rollback verify health, sign-in, a WebSocket edit and a sandbox run.

12 Final Requirements Traceability Matrix

This matrix links each requirement of Deliverable 1 (26 functional, 38 non-functional, with the identifiers the repository aligned to Deliverable 1 in CONCORD-68) to the Jira tasks that implement it, the pull requests that carry them, the tests or experiments that check it, and the status after this deliverable's measurements. Jira, pull-request and test columns come from the repository's own verification report (`docs/vv-report.md`) and the Jira export of 2 November 2026; the last two columns apply the Deliverable 3 re-run, and where that re-run replaced an older statement, the new text is shown.

Status key. Met / Verified : a test or a measurement meets the requirement as written. Covered : tests exercise the behaviour. Not measured : no measurement was taken. The chain for the two requirements Deliverable 1 marked critical is also given as a worked example below the tables.

12.1 Functional requirements

Req.	Jira	PR	Tests / experiments	Status	Deliverable 3 evidence
FR-01	CONCORD-23, CONCORD-24, CONCORD-66, CONCORD-73	#3, #6, #24	test_auth.py, test_security.py, test_gateway_authorization.py, test_adversarial_api.py	Verified	auth, token rejection and login throttling are covered by merged tests.
FR-02	CONCORD-27	#8	DashboardPage.test.tsx	Covered	Functional evidence exists; creation/ownership checked by pilot and client tests.
FR-03	CONCORD-29	#14	test_invitations.py	Covered	Invitation role/expiry checks covered by CI.
FR-04	CONCORD-29	#14	test_join_requests.py	Covered	Approve/reject/duplicate-request checks covered by CI.
FR-05	CONCORD-26	#17	test_members.py	Covered	Member-management and protected-owner checks covered by CI.
FR-06	CONCORD-25, CONCORD-73, CONCORD-76	#3, #7, #13, #22, #24	test_permissions.py, test_project_access_permissions.py, test_gateway_authorization.py, test_adversarial_api.py, test_execution.py	Verified	for the tested REST, WebSocket, route-inventory and browser Viewer scenarios.
FR-07	CONCORD-28, CONCORD-65	#1, #11, #24	test_files.py, test_folders.py, FileTree.test.tsx	Covered	create, rename, move, folder delete and path validation are covered;
FR-08	CONCORD-30, CONCORD-31, CONCORD-69	#3, #5, #9, #16, #21	test_concurrent_edits.py, test_realtime_gateway.py, test_gateway_convergence.py	Verified	
FR-09	CONCORD-33	#5, #9	test_awareness.py, Editor.test.tsx	Covered	Awareness behaviour covered; visual multi-client pilot evidence recorded.
FR-10	CONCORD-38, CONCORD-61	#12, #23	test_execution_state_machine.py, test_execution_events.py, test_worker_execution.py	Verified	accepted transitions are checked atomically and persisted with their events.
FR-11	CONCORD-35, CONCORD-38, CONCORD-73	#2, #12, #24	test_worker_execution.py, test_run_input.py, test_adversarial_api.py	Verified	language execution, compilation and stdin boundary checks are covered.
FR-12	CONCORD-39, CONCORD-41, CONCORD-67, CONCORD-76	#4, #22, #23, #27	test_execution.py, test_execution_events.py, ConsolePanel.test.tsx, e1_lifecycle.py	Verified	for browser shared status/output and ordered persisted events.

Req.	Jira	PR	Tests / experiments	Status	Deliverable 3 evidence
FR-13	CONCORD-40, CONCORD-60, CONCORD-75	#10, #12, #23	test_worker_execution.py, test_cancel_running.py, test_execution.py, e2_cancel.py	Verified	cancel removed the container after about 106 ms (E2)
FR-14	CONCORD-37, CONCORD-69	#10, #21	test_priority_queue.py, test_queue_atomic_claims.py, test_worker_execution.py	Verified	
FR-15	CONCORD-42, CONCORD-43, CONCORD-69	#3, #10, #21	test_priority_queue.py, test_queue_atomic_claims.py, e3_contention.py	Verified	priority and atomic-claim tests pass; E3 order owner, debugger, editor
FR-16	CONCORD-40, CONCORD-75	#10, #12, #23	test_orphan_sweep.py, test_failure_recovery.py, chaos.py	Verified	chaos recovery 315.522 s; orphan-sweep tests
FR-17	CONCORD-44, CONCORD-45	#2, #10, #15	test_dap_client.py, test_breakpoints.py, test_debug.py, test_debug_session.py	Covered	DAP and real sandbox worker checks passed; pilot Debug together evidence recorded.
FR-18	CONCORD-46, CONCORD-47, CONCORD-69, CONCORD-71	#2, #10, #12, #15, #21, #28	test_debug.py, test_debug_session.py, test_debug_single_slot.py, DebugPanel.test.tsx	Verified	for shared debug state and the slot claim: one of 12 concurrent starts succeeds and 11 are refused with 409. See NFR-P3 for latency.
FR-19	CONCORD-52	#9	test_snapshots.py, SnapshotsPanel.test.tsx	Covered	Snapshot safety/restore behaviour covered by backend/client tests.
FR-20	CONCORD-48	#4, #17, #19	test_llm_adapters.py, test_assistant_api.py	Covered	Provider adapters, encryption/configuration and owner restrictions covered.
FR-21	CONCORD-49, CONCORD-50, CONCORD-74	#4, #18, #19	test_agent_runner.py, test_assistant_api.py, AssistantPanel.test.tsx	Covered	Bounded tool contracts/human acceptance covered;
FR-22	CONCORD-51, CONCORD-64	#17, #19	test_assistant_api.py	Covered	Request audit covered;
FR-23	CONCORD-55, CONCORD-63	#19	test_metrics.py	Covered	Summary/recent metrics covered;
FR-24	CONCORD-51	#17, #19	test_llm_manager.py, test_assistant_api.py	Covered	Redis limit/reset behaviour covered with controlled dependencies.
FR-25	CONCORD-53	#16	test_versions.py, test_file_changes.py, HistoryModal.test.tsx	Covered	Version/restore and history UI covered.
FR-26	CONCORD-53	#16	test_search.py, SearchPanel.test.tsx	Covered	Cross-file text search and UI covered.

12.2 Non-functional requirements

Req.	Jira	PR	Tests / experiments	Status	Deliverable 3 evidence
NFR-U1	CONCORD-34, CONCORD-56, CONCORD-76	#3, #5, #16, #22	test_navigation.py, test_collaboration.py, test_execution.py	Covered	browser tests run in Chromium
NFR-U2	CONCORD-33	#5, #9	test_awareness.py, Editor.test.tsx	Covered	Behaviour covered; confirm colours/names/selections visually in pilot.

Req.	Jira	PR	Tests / experiments	Status	Deliverable 3 evidence
NFR-U3	CONCORD-38, CONCORD-34	#5, #12, #16	test_worker_execution.py, ConsolePanel.test.tsx	Covered	Error results/rendering covered; inspect representative line references in pilot.
NFR-U4	CONCORD-34, CONCORD-56	#3, #5, #16	SessionPage.test.tsx, test_navigation.py	Not measured	no timed novice study was run
NFR-P1	CONCORD-31, CONCORD-57, CONCORD-69	#5, #9, #16, #21, #30	load_test.py, test_gateway_convergence.py	Met	p99 11.3 ms (10 editors), 50.1 ms (25); 18000/18000 and 120000/120000 delivered
NFR-P2	CONCORD-55, CONCORD-57, CONCORD-67	#19, #27, #30	load_test.py, test_metrics.py, e4_burst.py	Met	p95 0.488 s and 0.625 s in load runs, 0.96 s in E4; earlier PR #30 runs on another machine gave 1.23 to 1.63 s (not met)
NFR-P3	CONCORD-46, CONCORD-67, CONCORD-71	#2, #15, #27, #28	test_debug_session.py, e5_debug_step.py	Met	40 steps: median 136.8 ms, p95 146.3 ms, max 146.8 ms
NFR-P4	CONCORD-34	#5, #16	Editor.test.tsx	Not measured	no frame-time profile was taken
NFR-S1	CONCORD-36	#10	test_sandbox_security.py	Covered	Memory-limit and isolation mechanisms are covered by the Linux worker CI suite.
NFR-S2	CONCORD-36	#10	test_sandbox_security.py	Covered	Configured quota supported by container checks;
NFR-S3	CONCORD-34, CONCORD-72	#5, #16, #28		Met	production bundle 722 kB gzip (JS 707.6 + CSS 14.1 + HTML 0.5) against 2.5 MB
NFR-S4	CONCORD-53	#16	test_version_pruning.py	Covered	Retention/pruning rules covered by unit/integration checks.
NFR-D1	CONCORD-54, CONCORD-70	#25	test_health.py	Not measured	no uptime sample was taken
NFR-D2	CONCORD-32, CONCORD-75	#3, #23	test_checkpoint.py, test_failure_recovery.py	Covered	restart restores the last checkpoint;
NFR-D3	CONCORD-30, CONCORD-31, CONCORD-69	#3, #5, #9, #16, #21	test_concurrent_edits.py, test_realtime_gateway.py, test_gateway_convergence.py	Verified	for tested update orderings and 10/25 concurrent clients through the real gateway.
NFR-D4	CONCORD-40, CONCORD-75	#10, #12, #23	test_orphan_sweep.py, test_failure_recovery.py, chaos.py	Met	worker-kill run: FAILED after 315.522 s (limit 360 s), sweeper event, no container left
NFR-D5	CONCORD-42, CONCORD-43, CONCORD-69	#3, #10, #21	test_priority_queue.py, test_queue_atomic_claims.py, test_debug_single_slot.py	Verified	
NFR-SEC1	CONCORD-36, CONCORD-62	#10, #29	test_sandbox_security.py, test_worker_workspace.py, sandbox.py, docker_adapter.py	Verified	for tested run, build, validation and debug containers, including zero capabilities and no-new-privileges.
NFR-SEC2	CONCORD-23, CONCORD-66	#6, #24	test_auth.py, test_adversarial_api.py	Covered	Hashing, login and throttling checks are covered;
NFR-SEC3	CONCORD-25, CONCORD-65, CONCORD-73, CONCORD-76	#3, #7, #13, #22, #24	test_permissions.py, test_realtime_gateway.py, test_adversarial_api.py, test_execution.py	Verified	for tested server-side role checks, path/route attacks and browser Viewer restrictions.
NFR-SEC4	CONCORD-49, CONCORD-74	#4, #18, #19	test_agent_runner.py	Covered	Project-scoped tool access covered;
NFR-ORG-ENV1	CONCORD-22, CONCORD-70	#3, #25	pilot.sh	Covered	Compose and production deployment configuration are merged;
NFR-ORG-ENV2	CONCORD-35	#2	test_worker_execution.py	Covered	Language image builds/execution passed in worker CI.

Req.	Jira	PR	Tests / experiments	Status	Deliverable 3 evidence
NFR-ORG-ENV3	CONCORD-21, CONCORD-22	#3	test_migrations_enums.py	Covered	Image/persistence configuration inspected; migration test covered.
NFR-ORG-OP1	CONCORD-54	#25	test_health.py, main.py	Verified	database, Redis and queue-depth checks cover degraded/unavailable responses and bounded dependency timeouts.
NFR-ORG-OP2	CONCORD-32	#3	test_checkpoint.py, test_failure_recovery.py	Covered	checkpoint persistence and restart restoration are covered;
NFR-ORG-OP3	CONCORD-41, CONCORD-61	#23	test_execution_events.py, test_failure_recovery.py	Verified	accepted transitions append ordered events; status and event commit or roll back together.
NFR-ORG-DEV1	CONCORD-22, CONCORD-56, CONCORD-72	#3, #28		Verified	lint, types, security, tests, build and e2e pass; JUnit/coverage artifacts and dependency audit reports are uploaded.
NFR-ORG-DEV2	CONCORD-72, CONCORD-77	#28		Met	backend line coverage 85.06%
NFR-ORG-DEV3	CONCORD-58, CONCORD-78	#1		Covered	Jira/PR records exist;
NFR-EXT-REG1	CONCORD-21, CONCORD-70	#3, #25	test_migrations_enums.py	Covered	TLS deployment configuration exists;
NFR-EXT-REG2	CONCORD-24, CONCORD-73	#3, #24	test_security.py, test_adversarial_api.py	Verified	the 24-hour expiry setting is present and expired tokens are rejected.
NFR-EXT-ETH1	CONCORD-49, CONCORD-50, CONCORD-74	#4, #18, #19	test_agent_runner.py, AssistantPanel.test.tsx, test_realtime_gateway.py	Covered	
NFR-EXT-ETH2	CONCORD-50, CONCORD-51, CONCORD-64	#4, #17, #19	test_assistant_api.py, test_file_changes.py	Covered	Request/change history covered;
NFR-EXT-LEG-ACC1	CONCORD-38, CONCORD-39, CONCORD-41	#4, #12, #23	test_worker_execution.py, test_metrics.py	Covered	execution timing/output records covered;
NFR-EXT-LEG-ACC2	CONCORD-51	#17, #19	test_assistant_api.py, test_llm_adapters.py	Covered	Provider token counts and request persistence covered.
NFR-EXT-LEG-SEC1	CONCORD-36, CONCORD-62	#10, #29	test_sandbox_security.py, sandbox.py, docker_adapter.py	Covered	Network isolation and capability/no-new-privileges checks are covered by Linux worker CI.
NFR-EXT-LEG-SEC2	CONCORD-36	#10	test_sandbox_security.py	Covered	Configured process limit supported by worker security checks.

12.3 Worked chains for the critical requirements

Req.	Story	Jira	PR	Test	Result
FR-14, FR-15, NFR-D5	US-04	CONCORD-37, CONCORD-42, CONCORD-43, CONCORD-69	#3, #10, #21	`test_priority_queue`, `test_queue_atomic_claims` (32-way race), E3 on real Redis	no slot assigned twice; start order owner, debugger, editor
FR-10, FR-11	US-03	CONCORD-38, CONCORD-61	#12, #23	state-machine unit tests, `test_execution_events`, worker tests with real containers	illegal transitions refused; D3 before/after
FR-16	US-03	CONCORD-40, CONCORD-75	#23	orphan-sweep tests, `chaos.py`	FAILED after 315.5 s, 0 containers left
FR-17, FR-18	US-05	CONCORD-44 to CONCORD-47, CONCORD-71	#15, #28	DAP client tests, `test_debug_single_slot`, E5	1 of 12 starts succeeds; step 136.8 ms median
FR-08, NFR-P1	US-02	CONCORD-30, CONCORD-31, CONCORD-69	#9, #21	`test_gateway_convergence` [10, 25], load test	18 000/18 000 and 120 000/120 000 delivered
FR-06	US-02	CONCORD-25, CONCORD-73, CONCORD-76	#7, #22, #24	adversarial suite, browser Viewer test	across the four roles for the routes tested
FR-21, FR-22	US-07	CONCORD-49, CONCORD-50, CONCORD-64, CONCORD-74	#18, #19	`test_agent_runner`, `test_assistant_api`	request audit and tool scoping covered

12.4 Jira task to pull request

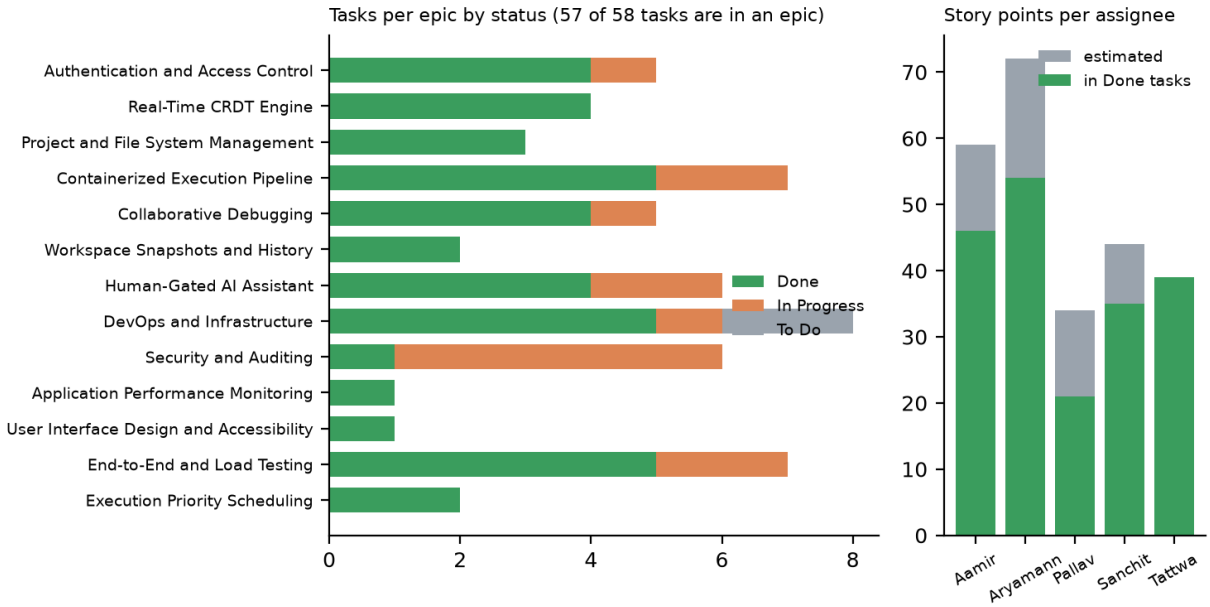
Key	Summary	Epic	Status	Pts	PRs
CONCORD-21	Design and initialize the PostgreSQL schema with Alembic migrations.	DevOps and Infrastructure	Done	5	#3
CONCORD-22	Containerize the full system with Docker Compose and set up the GitHub Actions pipeline for lint	DevOps and Infrastructure	Done	5	#3
CONCORD-23	As a user, I can sign up and log in with email and password, with passwords stored only as bcryp	Authentication and Access Cont	Done	5	#6
CONCORD-24	As a user, I receive an HS256 JWT on login that is validated on every REST request and WebSocket	Authentication and Access Cont	Done	5	#3
CONCORD-25	Enforce the Viewer, Editor, Debugger and Owner role hierarchy on every REST request and WebSocke	Authentication and Access Cont	Done	3	#3, #7
CONCORD-26	As an Owner, I can add members, change their roles and remove them, while my own role stays fixe	Authentication and Access Cont	Done	3	#17
CONCORD-27	As a user, I can create a project and automatically become its Owner.	Project and File System Manage	Done	3	#8
CONCORD-28	As an Editor, I can create, rename and move files and folders and delete folders, and every memb	Project and File System Manage	Done	5	#1, #1
CONCORD-29	As an Editor, I can generate a seven-day invitation link with a role no higher than my own, and	Project and File System Manage	Done	5	#14
CONCORD-30	Integrate the Yjs-compatible CRDT library into the backend so each open file has an in-memory sh	Real-Time CRDT Engine	Done	5	#3
CONCORD-31	As an Editor, my edits propagate to all collaborators in under 300 ms (99th percentile) and conv	Real-Time CRDT Engine	Done	5	#5, #9
CONCORD-32	Checkpoint modified CRDT documents to PostgreSQL every 5 seconds and force a checkpoint on grace	Real-Time CRDT Engine	Done	3	#3
CONCORD-33	As a collaborator, I can see every other member's live cursor with a name and a distinct colour.	Real-Time CRDT Engine	Done	3	#5, #9
CONCORD-34	Build the IDE-style React client with a file tree, tabbed Monaco editor, console and side panels	User Interface Design and Acce	Done	8	#5, #1
CONCORD-35	Build one sandbox Docker image per language (Python 3.12, Node.js 20, C++17 with g++) for use by	Containerized Execution Pipeli	Done	5	#2
CONCORD-36	Harden sandbox containers: non-root user, no network, read-only file system, 128 MB memory, 0.5	Security and Auditing	Done	5	#10

Key	Summary	Epic	Status	Pts	PRs
CONCORD-37	Push run requests onto a Redis queue consumed by a bounded pool of worker processes that each ru	Containerized Execution Pipeli	Done	5	#10
CONCORD-38	As an Editor, I can run a file in a sandbox that moves through QUEUED, STARTING, COMPILING, RUNN	Containerized Execution Pipeli	Done	8	#12
CONCORD-39	Broadcast every execution status change, and the final output, error stream, exit code and durat	Containerized Execution Pipeli	Done	5	#4
CONCORD-40	As an Editor, I can cancel a queued execution, and a background sweeper marks executions stuck f	Containerized Execution Pipeli	Done	5	#10, #
CONCORD-41	Record every execution state change in an append-only execution-event log.	Security and Auditing	In Progress	3	#23
CONCORD-42	Replace the FIFO Redis list with a priority-ordered queue so a free slot goes to the highest-pri	Execution Priority Scheduling	Done	8	#3
CONCORD-43	Implement and unit-test the priority score (role weight, ageing wait time and compilation cost).	Execution Priority Scheduling	Done	5	#10
CONCORD-44	Implement a Debug Adapter Protocol client inside the worker that talks to the debugger over the	Collaborative Debugging	Done	8	#2, #1
CONCORD-45	As a Debugger, I can set and clear breakpoints and control execution with continue, next, step i	Collaborative Debugging	Done	5	#2, #1
CONCORD-46	As a project member, I see the same paused line, call stack and variables as everyone else in th	Collaborative Debugging	Done	5	#2, #1
CONCORD-47	Limit each project to one active debug session and end it automatically after 600 seconds or 180	Collaborative Debugging	Done	3	#2, #1
CONCORD-48	Add a provider-independent LLM interface with Mistral and HuggingFace implementations and let on	Human-Gated AI Assistant	Done	5	#4, #1
CONCORD-49	Implement a bounded tool-calling agent loop (maximum six steps, five tools) that runs only on th	Human-Gated AI Assistant	Done	8	#4, #1
CONCORD-50	As an Editor, I receive AI code changes as side-by-side proposals that are only applied when I e	Human-Gated AI Assistant	Done	5	#4, #1
CONCORD-51	Record every AI request with its mode, prompt, response, provider, token counts and latency, and	Human-Gated AI Assistant	Done	3	#17, #
CONCORD-52	As an Editor, I can create a named snapshot of the whole project and restore it, with a safety s	Workspace Snapshots and Histor	Done	5	#9
CONCORD-53	As a member, I can view differences between versions of a file, an Editor can restore an earlier	Workspace Snapshots and Histor	Done	5	#16
CONCORD-54	Extend the backend health-check endpoint to report PostgreSQL connectivity, Redis connectivity a	DevOps and Infrastructure	Done	2	#25
CONCORD-55	Add a metrics page showing execution status counts, runs in the last hour and recent events.	Application Performance Monito	Done	3	#19
CONCORD-56	Build the end-to-end test suite covering login, collaborative editing, execution, debugging and	End-to-End and Load Testing	Done	5	#3
CONCORD-57	Run a load test with ten concurrent editors and five simultaneous run requests and check the lat	End-to-End and Load Testing	Done	5	#30
CONCORD-58	README and API documentation	DevOps and Infrastructure	Done	3	#1
CONCORD-59	Fixing bugs: application does not start from a fresh clone of main	none	Done	none	#20
CONCORD-60	Bug: cancelling a running execution does not stop its sandbox container	Containerized Execution Pipeli	In Progress	5	#23
CONCORD-61	Bug: execution status changes are not checked against the allowed-transition table	Containerized Execution Pipeli	In Progress	3	#23
CONCORD-62	Bug: sandbox containers keep Linux capabilities and can gain new privileges	Security and Auditing	In Progress	2	#29
CONCORD-63	Bug: metrics endpoints are readable without logging in and show every project's runs	Security and Auditing	In Progress	3	none
CONCORD-64	Bug: AI tool calls are not stored in the request audit record	Human-Gated AI Assistant	In Progress	3	none
CONCORD-65	Bug: file and folder paths accept "..", absolute paths and empty segments	Security and Auditing	In Progress	3	#24
CONCORD-66	Bug: login does not limit repeated failed attempts	Authentication and Access Cont	In Progress	3	#24
CONCORD-67	Reproducible experiment scripts for the core workflow (E1 to E5)	End-to-End and Load Testing	Done	3	#27
CONCORD-68	Align the repository's requirement IDs with Deliverable 1 numbering	DevOps and Infrastructure	In Progress	2	#26
CONCORD-69	Concurrency test suite: WebSocket convergence at 10 and 25 clients, atomic queue claims, single	End-to-End and Load Testing	Done	5	#21
CONCORD-70	Restore the production deployment configuration and write the deployment guide	DevOps and Infrastructure	Done	5	#25

Key	Summary	Epic	Status	Pts	PRs
CONCORD-71	Reduce debug-step propagation latency below 150 ms	Collaborative Debugging	In Progress	5	#28
CONCORD-72	Publish test reports, coverage reports and dependency audits from CI	End-to-End and Load Testing	In Progress	3	#28
CONCORD-73	Adversarial API test suite: token tampering, cross-project access, path traversal and oversized	Security and Auditing	In Progress	5	#24
CONCORD-74	Adversarial tests for the AI agent: prompt injection, tool abuse and audit records	Human-Gated AI Assistant	In Progress	3	none
CONCORD-75	Failure and recovery tests and the worker-kill script	End-to-End and Load Testing	In Progress	5	#23
CONCORD-76	Browser system tests for running, cancelling, metrics and viewer restrictions	End-to-End and Load Testing	Done	3	#22
CONCORD-77	Deliverable 3 integration, coverage gate and release	DevOps and Infrastructure	To Do	3	none
CONCORD-78	Final Jira dashboard and Sprint 2 closeout	DevOps and Infrastructure	To Do	2	none

13 Final Jira Dashboard and Project Report

All figures are read from Jira project CONCORD on 2 November 2026 (`https://cs455-project.atlassian.net`) and from the pull requests of `https://github.com/aamirepe/Concord---CS455-Course-Project`. The dashboard shown is drawn from that export; the Jira board itself was not modified. The final dashboard and sprint closeout are the open task CONCORD-78.



13.1 Totals

Measure	Tasks	Story points	Note
All tasks (13 epics, 2 template sub-tasks excluded)	58	248	
Done	42	195	79 % of points
In Progress	14	48	11 of them (39 points) already have a merged pull request
To Do	2	5	CONCORD-77, CONCORD-78
Sprint 1 (35 tasks)	35	172	all Done; the sprint's state in Jira is still "future" (never started or closed)
Sprint 2 (22 tasks, active, 19 Oct to 6 Nov)	22	76	6 Done (23 points), 14 In Progress, 2 To Do
Not in a sprint	1	0	CONCORD-59, a Done defect ticket without an estimate

Sprint 2's goal, as recorded on the board, is: *fix the recorded defects, complete the missing features, and add concurrency, adversarial, failure-recovery and load tests with reports from CI.*

13.2 Epics

Jira	Epic	Done	In prog.	To do	Points	Points done
CONCORD-6	Authentication and Access Control	4	1	0	19	16
CONCORD-7	Real-Time CRDT Engine	4	0	0	16	16
CONCORD-8	Project and File System Management	3	0	0	13	13
CONCORD-9	Containerized Execution Pipeline	5	2	0	36	28
CONCORD-10	Collaborative Debugging	4	1	0	26	21
CONCORD-11	Workspace Snapshots and History	2	0	0	10	10
CONCORD-12	Human-Gated AI Assistant	4	2	0	27	21
CONCORD-13	DevOps and Infrastructure	5	1	2	27	20
CONCORD-14	Security and Auditing	1	5	0	21	5
CONCORD-15	Application Performance Monitoring	1	0	0	3	3
CONCORD-16	User Interface Design and Accessibility	1	0	0	8	8
CONCORD-17	End-to-End and Load Testing	5	2	0	29	21
CONCORD-18	Execution Priority Scheduling	2	0	0	13	13

13.3 Work by person

Assignee	Tasks	Done	Points	Points done
Aryamann Srivastava	15	10	72	54
Aamir Ahmad	15	11	59	46
Sanchit Arora	11	8	44	35
Tattwa Shiwani	8	8	39	39
Pallav Rastogi	9	5	34	21

Merged pull requests by author: 30 in total, about 39 000 lines added and 800 removed, 173 commits in the pull requests. Jira's "In Progress" counts for Aryamann Srivastava, Pallav Rastogi and Aamir Ahmad include tickets whose pull requests are merged; the tables above therefore understate completed work for them.

14 AI Engineering Log

14.1 Method and scope

This log covers the work that produced *this document*: the evidence runs, the extraction of Jira and GitHub data, and the drafting of the report. It was carried out by one team member (Aryamann Srivastava) with Claude Code in the Claude desktop app (model Sonnet 5.5), which read files, ran commands and drafted text on the member's machine. The AI proposed and executed; the member set the task, supplied the constraints, and is the person who accepts or rejects the result. As in Deliverable 2, no AI output entered the document without a check against a source (a log, a Jira field, a file in the repository), and each entry records what was wrong and what changed. Teammates' AI use for their own Deliverable 3 pull requests is recorded in those pull requests (for example PR #28 and PR #30 name the assistant used); their entries are theirs to add here and are not summarised on their behalf.

Constraints given to the assistant. The existing repository checkout, the Jira project and the submitted deliverables were not to be changed; everything new goes into one new folder (`Deliverables/deliverable-3`) so it can be deleted to roll back; a checksum list of the 111 existing files in `Deliverables` was taken first and re-checked afterwards (111 of 111 unchanged), and the repository checkout's `HEAD` and clean status were recorded the same way. Measurements ran on a clean clone and in separate Compose and container names, not in the developer's stack. Only the official repository is described; no other repository appears in the document.

14.2 Entries

AI-D3-01: Scoping the deliverable from the brief and the earlier deliverables

Prompt	**Context:** the Deliverable 3 brief (14 items), Deliverables 0 to 2, the repository and the Jira export. **Role:** requirements and test engineer. **Task:** read everything, work out what each item needs, produce all of it. **Constraints:** existing material unharmed and rollback possible (added by the member mid-task); nothing fabricated. **Expected output:** a plan, then the artifacts.
AI output	A plan of measurements (test and coverage re-run, concurrency, load, failure, security scans, before/after regression runs), a list of facts to pull from Jira and GitHub, and the folder layout.
Evaluation	Compared with the brief, every item had a source or a measurement planned. The member's constraint changed the plan: baseline checksums and a local rollback repository inside the new folder were added.
Errors found	None in the plan itself; the plan did not yet say how to avoid disturbing the developer's running stack.
Changes made	Added the isolated Compose project, disposable database containers and the checksum check.
Final artifact	Sections 1.3 and the folder `D3_source/`.

AI-D3-02: Test, coverage and static-analysis re-run

Prompt	**Task:** run backend, worker and frontend suites with coverage, linters, type checks, bandit and dependency audits on a clean clone of `main`.
AI output	Three runner scripts and the result files; a first frontend run reported 115 failed tests of 230.
Evaluation	The 115 failures all read `Cannot read properties of undefined (reading 'clear')` in one `beforeEach`. The assistant checked CI's Node version (20) against the laptop's (26) and re-ran in a Node 20 container: 230 of 230 passed.
Errors found	The first result was an environment mismatch, not a product defect; reporting "115 failing frontend tests" would have been wrong.
Changes made	Frontend gates now run in a Node 20 container; the mismatch is noted in the log only.
Final artifact	Sections 2, 3; `D3_source/scripts/run_frontend_checks.sh`.

AI-D3-03: Experiments, load and chaos runs

Prompt	**Task:** run E1 to E5, the 10- and 25-editor load tests and the worker-kill test against an isolated stack and keep the raw output.
AI output	Result files and a summariser that computes medians and percentiles from the raw JSON.
Evaluation	Numbers were cross-checked against the scripts' own summaries (load test) and against CI for counts and coverage; the debug-step and queue-wait numbers were compared with those recorded in PR #28 and PR #30.
Errors found	The first load-test attempt crashed with WebSocket close code 1008: the script's default WebSocket address is port 8000, the developer's other stack, not the isolated one on 18000. Those connections carried tokens that stack does not know and were refused; no data was written there.
Changes made	Re-run with an explicit `--ws`; the run script now passes it. The NFR-P2 result differs from the earlier recorded runs, so the document explains the queue arithmetic instead of replacing the earlier numbers.
Final artifact	Sections 4, 6, 7; `evidence/*.json`.

AI-D3-04: Showing that each fix removes its defect (before/after runs)

Prompt	**Task:** for each defect-fix pull request, run its regression tests on the code before the fix and after it.
AI output	A script that checks out the base commit, overlays the pull request's test files, runs them, then runs them on the merge commit.
Evaluation	The summary lines (counts, failing test names, assertion text) were read, not just the exit code.
Errors found	(1) The first run was invalid: older commits depend on `python-jose`, which the shared environment lacked, so even the "after" runs failed on `import jose`. (2) For the login-limit and state-machine fixes the "before" run could only report an import error because the fix creates the module; that proves the tests cannot run on the old tree, not that the behaviour failed.
Changes made	Per-commit dependency install; an extra before/after case on the two path-validation test files, which do run on both trees (14 failed, 14 passed before; 28 passed after). The report states which cases are import errors.
Final artifact	Section 8; `evidence/regression_before_after.log`.

AI-D3-05: Extracting Jira and GitHub facts

Prompt	**Task:** read all tickets, sprints, pull requests, reviews and CI runs; produce counts and the requirement-to-Jira-to-pull-request links. Read-only.
AI output	JSON extracts and the generated tables.
Evaluation	Totals were reconciled (points 195 + 48 + 5 = 248; 57 tasks in epics + 1 without = 58; 53 + 5 = 58 for pull-request links). The Jira bug descriptions were read to word each defect's cause.
Errors found	(1) Story points and sprint came out empty at first because the connector returns them under a mapped-field block, not under their field ids. (2) A bulk GitHub query failed on a node limit and was redone per pull request. (3) My first draft called the login limiter's fail-open behaviour a defect; CONCORD-66 requires it, so it was reclassified as a design decision.
Changes made	Extraction corrected; the draft's wording of five defect causes was replaced by the ticket text.
Final artifact	Sections 8, 12, 13; `evidence/jira_issues_2026-11-02.json`, `rtm_data.json`.

AI-D3-06: Drafting the report and verifying its claims

Prompt	**Task:** write each section from the evidence, with every number traceable.
AI output	The sections of this document and the table generators.
Evaluation	Each claim was checked against its source before the section was kept: test names by search in the repository, parameters (5 attempts, 900 s, 64 KiB) in code or Jira, totals recomputed.
Errors found	(1) A Gantt bar for Deliverable 0 had dates nobody had supplied; it was removed. (2) Two cost scenarios were wrong when recomputed (129.38 should be 129.40; the stopped-host scenario was 25.40, correct value 29.62). (3) A sentence said a regression test was "added in PR #23" without naming it; the two real test names replaced it. (4) A security cell said API keys are "never logged" without evidence; it was removed. (5) A claim that tickets lacked acceptance checkboxes was contradicted by the tickets' "Done when" lines and was deleted.
Changes made	Corrections applied as listed.
Final artifact	This document.

AI-D3-07: Costing inputs

Prompt	**Task:** find a current compute price and an LLM price for the costing section.
AI output	Two price figures with their sources.
Evaluation	The compute price agreed across three aggregators; the LLM price did not (one source differed by three times), so both are labelled as third-party and to be confirmed.
Errors found	The assistant could not open the vendors' price pages.
Changes made	Rows are tagged S (sourced), A (assumed) or M (measured), so a reader can see which numbers to re-check.
Final artifact	Section 9.

14.3 Human-in-the-loop statement

The assistant drafted and executed; it did not decide what the team accepts. Acceptance of this deliverable, the choice of what to fix before 6 November, and any change to Jira or to `main` are for the team. Nothing in the official repository, in Jira or in the local repository checkout was changed by the assistant in producing this document.

A Reproducing the Evidence

Everything below runs on a machine with Docker and Python 3.11 or newer. Replace `<clone>` with a clean clone of `main` and `<D3>` with the `D3_source` folder. The scripts do not touch a running development stack: they use their own Compose project name, ports and container names.

```
# 1. backend gates + 318 tests + coverage (PostgreSQL and Redis in throwaway containers)
<D3>/scripts/run_backend_checks.sh   <clone> <D3>/evidence
# 2. worker tests with real sandbox containers (needs the 3 sandbox images built)
<D3>/scripts/run_worker_checks.sh    <clone> <D3>/evidence /tmp/d3shared
# 3. frontend gates on Node 20
<D3>/scripts/run_frontend_checks.sh  <clone> <D3>/evidence
# 4. isolated stack on port 18000, seed, pilot check
cp <clone>/.env.example <clone>/.env      # then set SECRET_KEY and LLM_ENCRYPTION_KEY
<D3>/scripts/dc3.sh <clone> up -d --build postgres redis backend worker
<D3>/scripts/dc3.sh <clone> exec -T backend python /scripts/pilot.py seed
<D3>/scripts/dc3.sh <clone> exec -T backend python /scripts/pilot.py check
# 5. experiments E1-E5 and the two load tests (pass the isolated WebSocket address)
<D3>/scripts/run_experiments.sh <clone> <venv-python> <D3>/evidence
# 6. worker-kill test (about 6 minutes)
python <clone>/scripts/chaos.py --api http://localhost:18000 --ws ws://localhost:18000 \
   --compose-project concordd3 --project-id <id> --file-id <id>      # CONCORD_TOKEN set
# 7. before/after run of each defect fix
<D3>/scripts/run_regression_before_after.sh <clone> <D3>/evidence /tmp/d3shared2
# 8. tear down and rebuild the tables and figures
```

```
<D3>/scripts/dc3.sh <clone> down -v
python <D3>/scripts/summarize_evidence.py <D3>/evidence
python <D3>/scripts/gen_tables.py <D3> <clone>
python <D3>/scripts/make_figures.py <D3>
# 9. check every number, link, test name and path of the report against the evidence
pdftotext -layout <D3>/../Deliverable_3_Group_9.pdf /tmp/d3.txt
python <D3>/scripts/verify_report.py <D3> <clone> /tmp/d3.txt
```

B Files Delivered

Path (under Deliverables/deliverable-3/)	Contents
Deliverable_3_Group_9.pdf, .tex	This document and its source
D3_source/parts/	The source split by section; ./build.sh reassembles it
D3_source/scripts/	The runners, the summariser, the table and figure generators
D3_source/evidence/	Raw logs and JSON of every run quoted, coverage reports, Jira and GitHub extracts of 2 Nov
D3_source/figures/, generated/	Figures and the generated tables
D3_source/baseline/	Checksums of the 111 pre-existing files in Deliverables and the checkout's state before any work

C Rollback

All new material is under Deliverables/deliverable-3/. Deleting that folder removes it completely; the checksum list in baseline/deliverables_before.sha256 verifies that the 111 earlier files are unchanged (shasum -a 256 -c from the parent of Deliverables). The folder is also a local Git repository with a snapshot history of its own, so an earlier state of this document can be restored without touching anything outside it.